

Soccer Forecast Agent — Final Submission

Multi-Agent AI System for Premier League Match Forecasting and Alert Generation

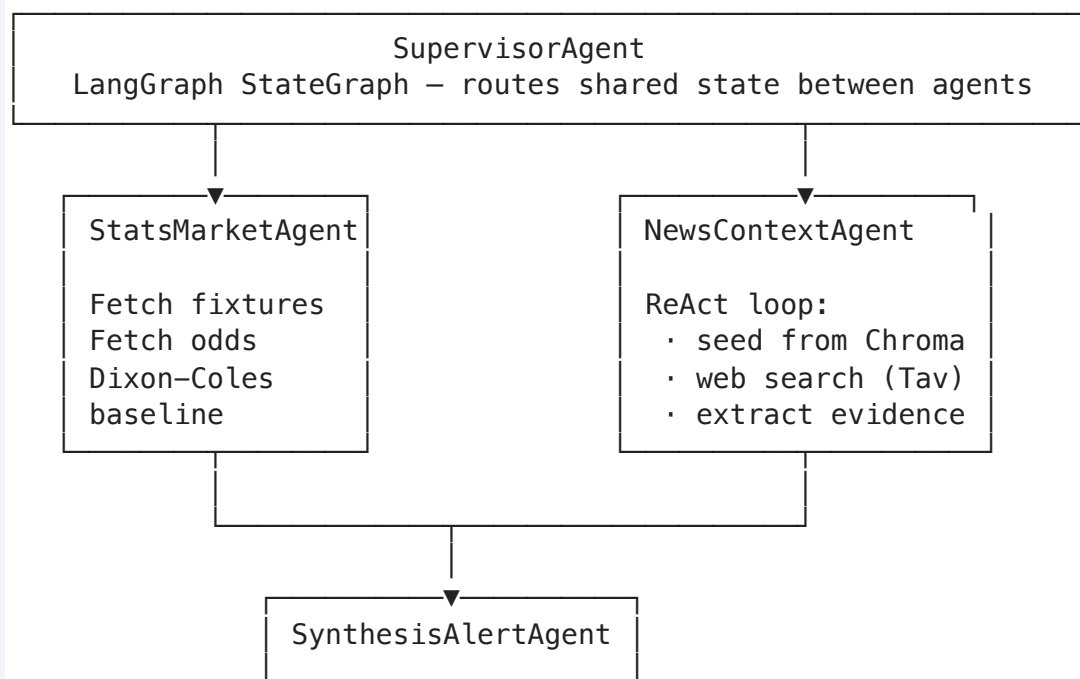
This notebook demonstrates the complete end-to-end *Soccer Forecast Agent*: a production-grade multi-agent AI system that monitors Premier League fixtures, computes probabilistic forecasts using a Dixon-Coles statistical model enhanced with recency-weighted form, gathers qualitative news evidence via a ReAct research loop, and fires styled HTML email alerts when a statistically meaningful edge is detected over market-implied probabilities.

Stack: Python 3.11 · LangGraph · OpenAI API · SQLite · ChromaDB · SentenceTransformers · The Odds API · Tavily Search

Note — API keys required to execute the live pipeline cells. Cells that call external APIs are marked with `# [LIVE]`. All other cells run offline against the local SQLite / ChromaDB stores.

System Architecture

The system follows a supervisor-based multi-agent pattern with four specialised agents orchestrated by a `LangGraph StateGraph`. All agents depend only on abstract `Protocol` interfaces — no concrete SDK import appears inside any agent class (**Dependency Inversion Principle**).



LLM adjudication
Log-odds adjustment
AlertGuard checks
HTML email dispatch

Agent	Responsibility	SOLID principle highlighted
SupervisorAgent	Orchestrates the match-loop LangGraph workflow	SRP
StatsMarketAgent	Fetches fixtures + odds, runs Dixon-Coles	OCP (strategy injection)
NewsContextAgent	ReAct think-act-observe research loop	DIP (LLMProvider protocol)
SynthesisAlertAgent	LLM synthesis, edge calculation, alert dispatch	ISP (narrow protocols)

Persistence: SQLite (structured — fixtures, forecasts, evidence) + ChromaDB (vector — article chunks)

1 · Environment Setup

Config loaded — LLM provider : openai model : gpt-4.1-mini
Embedding provider : huggingface
SQLite path : soccer_forecast.db
Resolved matches in DB : 385

2 · Core Data Models (Phase 1)

All structured data flows as typed dataclasses — no raw `dict` in any public interface.

```

=== Match ===
@dataclass
class Match:
    """A Premier League fixture with its current status and optional final score."""

    match_id: str
    competition: str
    home_team: str
    away_team: str
    kickoff_time: datetime
    status: str # "upcoming" | "live" | "resolved"
    final_score: str | None = None

=== MarketOdds ===
@dataclass
class MarketOdds:
    """Decimal odds for a match across winner and over/under markets at a point in time."""

    odds_id: str
    match_id: str
    timestamp: datetime
    home_win: float
    draw: float
    away_win: float
    over_2_5: float
    under_2_5: float
    winner_market_source: str = "Unknown source"
    goals_market_source: str = "Unknown source"
    market_sources_seen: list[str] = field(default_factory=list)

=== BaselineForecast ===
@dataclass
class BaselineForecast:
    """Probabilities produced by the statistical baseline, independent of market odds."""

    home_win: float
    draw: float
    away_win: float
    over_2_5: float
    under_2_5: float

```

Repository and LLM protocols

```

=== LLMProvider ===
class LLMProvider(Protocol):
    """Abstract interface for language model calls. Agents depend on this, never on a concrete SDK."""

    def chat(self, messages: list[dict[str, str]], **kwargs: Any) -> str:
        """Send a conversation and return the assistant reply as a string."""
        ...

    def chat_with_tools(
        self,
        messages: list[dict[str, str]],
        tools: list[dict],
        **kwargs: Any,
    ) -> dict:
        """Send a conversation with tool definitions; return the raw response dict including any tool calls."""
        ...

    def format_assistant_turn(self, response: dict) -> dict:
        """Convert a raw chat_with_tools response into a message dict suitable for appending to history."""
        ...

    def format_tool_result(self, tool_call_id: str, content: str) -> dict:
        """Build a tool-result message for appending to history after a tool was called."""
        ...

=== MatchRepository ===
class MatchRepository(Protocol):
    """Persistence contract for match fixtures."""

    def save_match(self, match: Match) -> None:
        """Persist a match, upserting on match_id."""
        ...

    def get_upcoming(self, competition: str) -> list[Match]:
        """Return all matches with status 'upcoming' for the given competition."""
        ...

    def get_recent_finished(self, team: str, competition: str, limit: int = 5) -> list[Match]:
        """Return the most recent finished matches for a team in a competition."""
        ...

    def update_status(self, match_id: str, status: str, final_score: str | None = None) -> None:
        """Update match status and optionally record the final score."""
        ...

=== VectorRepository ===
class VectorRepository(Protocol):

```

```

"""Persistence contract for the unstructured news vector store."""

def upsert(self, chunks: list[ArticleChunk]) -> None:
    """Embed and store article chunks, replacing any with the same chunk_id."""
    ...

def search(self, query: str, teams: list[str], top_k: int = 5) -> list[ArticleChunk]:
    """Return the top_k most semantically relevant chunks filtered by team names."""
    ...

```

3 · Alert Guardrails (Phase 1)

Six hard checks must pass before any alert fires.

```

@dataclass(frozen=True)
class AlertGuardConfig:
    """Tunable thresholds that define the alert quality bar."""

    min_evidence_count: int = 3
    min_avg_reliability: float = 0.5
    max_evidence_age_hours: int = 48
    min_unique_sources: int = 2
    min_edge_threshold: float = 0.05
    min_confidence_threshold: float = 0.60
    spam_window_hours: int = 6
    min_odds_delta: float = 0.05

```

```

Default thresholds:
min_evidence_count      : 3
min_avg_reliability     : 0.5
max_evidence_age_hours  : 48
min_unique_sources      : 2
min_edge_threshold      : 0.05
min_confidence_threshold : 0.6
spam_window_hours       : 6
min_odds_delta          : 0.05

```

4 · Statistical Baseline — Dixon-Coles Model (Phases 2 & 7)

Two baseline strategies are implemented and compared:

Strategy	Winner Brier ↓	Goals Brier ↓	Notes
SimpleBaselineStrategy	0.2207	0.2635	Rolling 5-game form
EnhancedBaselineStrategy	0.2202	0.2544	Recency-weighted form, blended xG

Strategy	Winner Brier ↓	Goals Brier ↓	Notes
DixonColesStrategy	0.2198	0.2673	MLE on full season, τ correction

Random baseline $\approx$ 0.222 (winner), 0.250 (goals). All three strategies beat random on the winner market.

Dixon-Coles is used as the default — it wins on the primary market and provides per-team attack/defence parameters.

Dixon-Coles fitted on 385 matches

Teams modelled : 23

Home advantage : 0.175 (log scale)

ρ (rho) : -0.137 (low-score correlation)

Team	Attack	Defence
Arsenal FC	+0.214	-0.151
Liverpool FC	+0.142	+0.321
Manchester City FC	+0.217	-0.162
Nottingham Forest FC	-0.209	+0.284
Chelsea FC	-0.020	+0.266
Burnley FC	-0.407	+0.698

Live baseline prediction for two upcoming fixtures

=====

Bournemouth vs Manchester City

=====

Outcome	Dixon-Coles	Enhanced
Home win	38.5%	38.2%
Draw	32.7%	26.0%
Away win	28.8%	35.8%
Over 2.5	37.5%	51.0%
Under 2.5	62.5%	49.0%

=====

Aston Villa vs Liverpool

=====

Outcome	Dixon-Coles	Enhanced
Home win	38.5%	38.2%
Draw	32.7%	26.0%
Away win	28.8%	35.8%
Over 2.5	37.5%	51.0%
Under 2.5	62.5%	49.0%

5 · News Context Agent — Dual Retrieval (Phase 3)

The NewsContextAgent runs a **ReAct (Reason + Act)** loop:

1. **Seed** — query ChromaDB for pre-indexed articles relevant to the match
2. **Think** — LLM reasons about what to search next
3. **Act** — dispatch `search_news` tool call to Tavily
4. **Observe** — extract structured `EvidenceItem` objects from results
5. **Stop** — when evidence count and reliability thresholds are met

```
/Users/lizethbuendia/Desktop/SoccerForecastAgent/.venv/lib/python3.11/site-packages/tqdm/
auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See h
ttps://ipywidgets.readthedocs.io/en/stable/user_install.html
```

```
from .autonotebook import tqdm as notebook_tqdm
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```

```
Loading weights: 0%|          | 0/103 [00:00<?, ?it/s]
```

```
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 6914.20it/s]
```

```
Vector store query : 'Bournemouth Manchester City injuries suspensions form'
Chunks retrieved   : 4
```

- [1] <https://www.transfermarkt.us/bournemouth-afc/sperrenundverletzungen/verein/989>
AFC Bournemouth – Suspensions and injuries | Transfermarkt # AFC Bournemouth. AFC Bournemouth U21AFC Bournemouth U21. AFC Bournemouth Jugend...
- [2] <https://www.newsnow.com/us/Sports/Soccer/Premier+League/Manchester+City/Injuries+and+Suspensions>
Manchester City Injuries and Suspensions News – NewsNow Latest news on Manchester City injuries and suspensions, covering squad updates, fit...
- [3] <https://www.premierinjuries.com/teams/afc-bournemouth>
AFC Bournemouth – Premier Injuries Which AFC Bournemouth players are injured? Details of all the current injuries, suspensions and absences,...
- [4] <https://www.newsnow.com/us/Sports/Soccer/Premier+League/Bournemouth/Injuries+and+Suspensions>
AFC Bournemouth Injuries & Suspensions news – NewsNow Dutch forward Justin Kluivert returns from knee surgery for Bournemouth ahead of World...

EvidenceItem — structured output of the research loop

```

@dataclass
class EvidenceItem:
    """A single piece of qualitative evidence gathered by the ReAct research agent."""

    evidence_id: str
    forecast_id: str
    source: str
    url: str
    timestamp: datetime
    summary: str
    direction: str          # "home_positive" | "away_positive" | "neutral" | "uncertainty"
    reliability_score: float # 0.0 – 1.0
    applies_to_market: str  # "winner" | "goals" | "both"

```

6 · Synthesis & Alert Agent (Phase 4)

`SynthesisAlertAgent` is the most complex agent in the system. It orchestrates five sub-steps:

1. **Evidence quality scoring** — deterministic score from reliability, source diversity, and count
2. **LLM synthesis decision** — structured JSON output with bounded adjustment labels and conviction score
3. **Bounded log-odds adjustment** — moves baseline probabilities in log-odds space, scaled by $\text{evidence_quality} \times \text{llm_conviction}$
4. **Edge calculation** — adjusted probability minus overround-normalised market-implied probability
5. **Dual guard** — `AlertGuard` hard checks AND LLM `alert_worthy` flag must both pass before an alert is sent

All tunable parameters are consolidated in `SynthesisTuning` and injected at construction time.

=== `SynthesisTuning` – all tuning knobs in one injectable dataclass ===


```

@dataclass(frozen=True)
class SynthesisTuning:
    """Typed tuning values for bounded synthesis adjustments and confidence scoring."""

    base_sensitivity: float = 0.15
    probability_floor: float = 0.01
    probability_ceiling: float = 0.99
    implied_probability_default: float = 0.0
    prior_odds_delta_default: float = 0.0
    evidence_quality_cap: float = 1.0
    evidence_quality_empty: float = 0.0
    source_bonus_per_unique_source: float = 0.05
    source_bonus_cap: float = 0.15
    count_bonus_per_item: float = 0.03
    count_bonus_cap: float = 0.15
    confidence_base: float = 0.15
    confidence_quality_weight: float = 0.45
    confidence_llm_weight: float = 0.35
    confidence_cap: float = 0.95
    rounding_precision: int = 4
    decision_evidence_limit: int = 6
    decision_temperature: float = 0.0
    decision_max_tokens: int = 350

=== SynthesisDecision – structured JSON output from the LLM ===
@dataclass
class SynthesisDecision:
    """Structured LLM adjudication over how the baseline should move and whether the signal is alert-worthy."""

    market_category: str
    recommended_market: str
    winner_adjustment: str
    goals_adjustment: str
    llm_conviction_score: float
    alert_worthy: bool
    rationale_points: list[str] = field(default_factory=list)
    risk_points: list[str] = field(default_factory=list)
    summary: str = ""

```

=== Bounded adjustment lookup tables ===

WINNER_ADJUSTMENTS (home_delta, away_delta in log-odds units):

strong_home	home	+1.0	away	-1.0
medium_home	home	+0.7	away	-0.7
light_home	home	+0.3	away	-0.3
neutral	home	+0.0	away	+0.0
light_draw	home	-0.2	away	-0.2
medium_draw	home	-0.4	away	-0.4
light_away	home	-0.3	away	+0.3
medium_away	home	-0.7	away	+0.7
strong_away	home	-1.0	away	+1.0

GOALS_ADJUSTMENTS (over_delta, under_delta in log-odds units):

strong_over	over	+1.0	under	-1.0
medium_over	over	+0.7	under	-0.7
light_over	over	+0.3	under	-0.3
neutral	over	+0.0	under	+0.0
light_under	over	-0.3	under	+0.3
medium_under	over	-0.7	under	+0.7
strong_under	over	-1.0	under	+1.0

Evidence quality scoring and confidence formula

```

def _evidence_quality_score(self, interpreted: list[InterpretedEvidence]) -> float:
    """Estimate deterministic evidence quality from reliability, source diversity, and count."""
    if not interpreted:
        return self._tuning.evidence_quality_empty
    avg_reliability = sum(item.reliability_score for item in interpreted) / len(interpreted)
    source_bonus = min(
        len({item.source for item in interpreted}) * self._tuning.source_bonus_per_unique_source,
        self._tuning.source_bonus_cap,
    )
    count_bonus = min(
        len(interpreted) * self._tuning.count_bonus_per_item,
        self._tuning.count_bonus_cap,
    )
    score = min(self._tuning.evidence_quality_cap, avg_reliability + source_bonus + count_bonus)
    return round(score, self._tuning.rounding_precision)

def _confidence_score(self, evidence_quality_score: float, llm_conviction_score: float) -> float:
    """Blend deterministic evidence quality with the synthesis LLM's conviction."""
    confidence = (
        self._tuning.confidence_base
        + (evidence_quality_score * self._tuning.confidence_quality_weight)
        + (llm_conviction_score * self._tuning.confidence_llm_weight)
    )
    return round(min(self._tuning.confidence_cap, confidence), self._tuning.rounding_precision)

```

Synthesis walkthrough — Bournemouth vs Manchester City

Synthesis walkthrough – Bournemouth vs Manchester City

Evidence quality score : 0.99
avg reliability : 0.75
source diversity bonus : +0.15
count bonus : +0.09

LLM decision
winner_adjustment : medium_home
goals_adjustment : neutral
llm_conviction_score : 0.70

Two-factor scale : 0.99 × 0.70 = 0.693

Probabilities

Outcome	Baseline	Adjusted
Home win	38.5%	40.1%
Draw	32.7%	32.6%
Away win	28.8%	27.3%

Edge = 40.1% – 22.1% = +18.0%
Confidence score : 84.0%

AlertGuard: edge  confidence  → alert_worthy check next
LLM alert_worthy=True → ALERT SENT 

7 · Full End-to-End Pipeline Run [LIVE – requires API keys]

The SupervisorAgent orchestrates the complete match-loop: for each upcoming fixture → StatsMarketAgent → (if edge) NewsContextAgent → SynthesisAlertAgent

Forecast + evidence tables cleared – starting fresh run.

```
Loading weights: 0%|          | 0/103 [00:00<?, ?it/s]
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 13558.04it/s]
Running supervisor – processing all upcoming PL fixtures...

[EDGE] Aston Villa vs Liverpool – baseline edge detected, running research
[EDGE] Manchester United vs Nottingham Forest – baseline edge detected, running research
[EDGE] Brentford vs Crystal Palace – baseline edge detected, running research
[EDGE] Everton vs Sunderland – baseline edge detected, running research
[EDGE] Leeds vs Brighton – baseline edge detected, running research
[EDGE] Wolves vs Fulham – baseline edge detected, running research
[EDGE] Newcastle vs West Ham – baseline edge detected, running research
[EDGE] Arsenal vs Burnley – baseline edge detected, running research
[EDGE] Bournemouth vs Manchester City – baseline edge detected, running research
[EDGE] Chelsea vs Tottenham – baseline edge detected, running research
```

Done. 10 forecasts generated.

Pipeline results summary

Match	Market	Edge	Conf	Alert
Leeds vs Brighton	home_win	+10.4%	69%	YES
Manchester United vs Nottingham Forest	draw	+9.6%	74%	YES
Everton vs Sunderland	away_win	+7.0%	74%	YES
Leeds vs Brighton	home_win	+10.6%	74%	no
Aston Villa vs Liverpool	home_win	+5.5%	72%	no
Newcastle vs West Ham	away_win	+0.8%	74%	no
Chelsea vs Tottenham	home_win	-5.4%	76%	no
Chelsea vs Tottenham	home_win	-6.2%	71%	no
Newcastle vs West Ham	home_win	-6.4%	71%	no
Everton vs Sunderland	home_win	-12.2%	72%	no
Brentford vs Crystal Palace	home_win	-14.5%	75%	no
Brentford vs Crystal Palace	home_win	-14.6%	72%	no
Manchester United vs Nottingham Forest	home_win	-17.6%	76%	no
Wolves vs Fulham	away_win	-19.9%	76%	no
Wolves vs Fulham	away_win	-19.9%	76%	no
Bournemouth vs Manchester City	away_win	-24.4%	77%	no
Bournemouth vs Manchester City	away_win	-24.6%	74%	no
Arsenal vs Burnley	home_win	-47.3%	80%	no
Arsenal vs Burnley	home_win	-47.5%	76%	no

Alerts sent : 3 / 19 forecasts generated

8 · HTML Alert Email — Rendered Inline

Leeds vs Brighton

📅 Sunday, 17 May 2026 · 14:00 UTC

VALUE SIGNAL

Leeds win +10.4%

We estimate Leeds win at 39.6%, vs market-implied 29.2% — a +10.4% gap.

PROBABILITIES

Market	Baseline	Adjusted
Leeds win	38.5%	39.6%
Draw	32.7%	32.6%
Brighton win	28.8%	27.8%
Over 2.5 goals	37.5%	37.5%
Under 2.5 goals	62.5%	62.5%

CONFIDENCE



ODDS SOURCE

William Hill via The Odds API (also seen: William Hill, Betfred, Sky Bet)

RATIONALE

Leeds' home unbeaten run and Brighton's key injuries suggest a moderate edge to Leeds for the win, while goal expectations remain balanced. Key reasons: Leeds have a strong home unbeaten streak against Brighton, indicating home advantage.; Brighton's multiple key injuries likely weaken their squad more than Leeds' absences.; Motivation for Leeds to finish home season strongly supports home win bias. Main risks: Leeds also face significant injuries and suspensions, which could offset home advantage.; Brighton's fewer absences and potential tactical adjustments may limit Leeds' edge. Recommended market: home_win; adjusted edge: +10.4%.

This is decision support only. Not a guarantee of profit. Probabilities are estimates, not certainties.

Rendered alert: Leeds vs Brighton edge=+10.4%

9 · Model Validation — Backtest (Phase 7)

Chronological 70/30 train/test split. Dixon-Coles is fit on the training window only; all three strategies are evaluated on unseen test matches. Brier score ↓ is better. Random baseline ≈ 0.222 (winner), 0.250 (goals).

.2831	0.3339	0.49	0.54	0.2833	0.2904	0.20	0.40	0.2505	0.1567		
2026-04-12	Chelsea FC			Manchester City FC		0-3	A	0.22	0.60	0.1177	0.16
00 ✓ 0.23	0.55	0.1190	0.1996 ✓ 0.32	0.43	0.2086	0.3227✓					
2026-04-13	Manchester United			Leeds United FC		1-2	A	0.61	0.42	0.3987	0.33
39 0.60	0.44	0.3890	0.3148	0.56	0.69	0.3295	0.0992				
2026-04-18	Brentford FC			Fulham FC		0-0	D	0.41	0.56	0.2749	0.30
87 0.41	0.52	0.2750	0.2752	0.51	0.62	0.2901	0.3798				
2026-04-18	Newcastle United F			AFC Bournemouth		1-2	A	0.35	0.53	0.1892	0.21
78 ✓ 0.33	0.51	0.1779	0.2398 ✓ 0.54	0.64	0.3243	0.1331					
2026-04-18	Leeds United FC			Wolverhampton Wand		3-0	H	0.33	0.42	0.2301	0.33
39 0.36	0.44	0.2051	0.3148	0.65	0.54	0.0617	0.2075✓				
2026-04-18	Tottenham Hotspur			Brighton & Hove Al		2-2	D	0.12	0.51	0.3147	0.23
90 0.12	0.50	0.3158	0.2540	0.32	0.54	0.2521	0.2141				
2026-04-18	Chelsea FC			Manchester United		0-1	A	0.25	0.71	0.1310	0.50
57 ✓ 0.25	0.62	0.1308	0.3901 ✓ 0.55	0.56	0.3397	0.3109					
2026-04-19	Nottingham Forest			Burnley FC		4-1	H	0.58	0.60	0.0915	0.16
00 ✓ 0.58	0.55	0.0896	0.1996 ✓ 0.45	0.45	0.1528	0.3069✓					
2026-04-19	Aston Villa FC			Sunderland AFC		4-3	H	0.23	0.51	0.3062	0.23
90 0.23	0.50	0.3043	0.2540	0.51	0.34	0.1262	0.4301✓				
2026-04-19	Everton FC			Liverpool FC		1-2	A	0.41	0.73	0.2267	0.07
11 0.40	0.64	0.2220	0.1304	0.38	0.43	0.2453	0.3223				
2026-04-19	Manchester City FC			Arsenal FC		2-1	H	0.36	0.60	0.2086	0.16
00 0.37	0.55	0.2037	0.1996 ✓ 0.36	0.36	0.2040	0.4058✓					
2026-04-20	Crystal Palace FC			West Ham United FC		0-0	D	0.44	0.60	0.2775	0.36
00 0.44	0.55	0.2773	0.3060	0.55	0.52	0.2912	0.2662				
2026-04-21	Brighton & Hove Al			Chelsea FC		3-0	H	0.57	0.53	0.0938	0.21
78 ✓ 0.58	0.51	0.0885	0.2398 ✓ 0.35	0.48	0.2144	0.2681✓					
2026-04-22	AFC Bournemouth			Leeds United FC		2-2	D	0.36	0.38	0.2738	0.38
71 0.36	0.41	0.2739	0.3477	0.49	0.72	0.3002	0.0766				
2026-04-22	Burnley FC			Manchester City FC		0-1	A	0.09	0.58	0.0662	0.33
39 ✓ 0.09	0.54	0.0650	0.2904 ✓ 0.10	0.58	0.0511	0.3394✓					
2026-04-24	Sunderland AFC			Nottingham Forest		0-5	A	0.30	0.60	0.1544	0.16
00 ✓ 0.28	0.55	0.1485	0.1996 ✓ 0.49	0.32	0.3459	0.4640					
2026-04-25	Fulham FC			Aston Villa FC		1-0	H	0.33	0.60	0.2286	0.36
00 0.32	0.55	0.2350	0.3060	0.31	0.47	0.2397	0.2217				
2026-04-25	Liverpool FC			Crystal Palace FC		3-1	H	0.36	0.51	0.2091	0.23
90 0.37	0.50	0.2013	0.2540 ✓ 0.49	0.48	0.1330	0.2733✓					
2026-04-25	West Ham United FC			Everton FC		2-1	H	0.40	0.51	0.1782	0.23
90 ✓ 0.41	0.50	0.1746	0.2540 ✓ 0.28	0.46	0.2589	0.2963					
2026-04-25	Wolverhampton Wand			Tottenham Hotspur		0-1	A	0.49	0.67	0.2891	0.44
45 0.47	0.60	0.2730	0.3552	0.23	0.41	0.1524	0.1656✓				

Test matches evaluated : 116 | Skipped : 0 (insufficient history)

Metric	Simple	Enhanced	Vs S	DC	Vs S
Winner Brier (random ≈ 0.222)	0.2207	0.2202	↓0.0005	0.2198	↓0.0009
Goals Brier (random ≈ 0.250)	0.2635	0.2544	↓0.0091	0.2673	↑0.0038
Accuracy (most-likely outcome)	43.1%	43.1%		44.0%	

10 · Test Suite

35 unit tests across all system layers. Fakes replace external dependencies — no mocking of internals.

===== test session starts =====

platform darwin -- Python 3.11.15, pytest-9.0.3, pluggy-1.6.0 -- /Users/lizethbuendia/Desktop/SoccerForecastAgent/.venv/bin/python

cachedir: .pytest_cache

rootdir: /Users/lizethbuendia/Desktop/SoccerForecastAgent

configfile: pyproject.toml

plugins: langsmith-0.7.36, anyio-4.13.0

collecting ... collected 43 items

tests/test_alert_guard.py::test_alert_guard_passes_with_recent_diverse_reliable_evidence PASSED [2%]

tests/test_alert_guard.py::test_alert_guard_collects_multiple_failure_reasons PASSED [4%]

tests/test_alert_guard.py::test_alert_guard_handles_naive_datetimes_without_crashing PASSED [6%]

tests/test_alert_guard.py::test_alert_guard_uses_default_config_values PASSED [9%]

tests/test_baseline.py::test_simple_baseline_returns_normalized_probabilities PASSED [11%]

tests/test_baseline.py::test_simple_baseline_favors_stronger_home_side PASSED [13%]

tests/test_baseline.py::test_simple_baseline_keeps_nonzero_away_win_probability_for_weak_side PASSED [16%]

tests/test_chroma_repository.py::test_chroma_repository_uses_embedding_provider_for_upsert_and_query PASSED [18%]

tests/test_config.py::test_config_defaults_embedding_provider_and_model PASSED [20%]

tests/test_config.py::test_config_reads_llm_model_override PASSED [23%]

tests/test_embedding_retrieval_fixture.py::test_sample_fixture_contains_six_articles PASSED [25%]

tests/test_embedding_retrieval_fixture.py::test_retrieval_fixture_surfaces_expected_team_and_topic_matches PASSED [27%]

tests/test_features.py::test_feature_extractor_builds_context_with_last_five_results PASSED [30%]

tests/test_features.py::test_feature_extractor_uses_default_average_for_empty_series PASSED [32%]

tests/test_ingester.py::test_article_ingester_skips_blank_and_duplicate_urls PASSED [34%]

tests/test_ingester.py::test_article_ingester_chunking_uses_overlap PASSED [37%]

tests/test_main.py::test_local_mcp_tool_client_dispatches_search_news PASSED [39%]

tests/test_main.py::test_local_mcp_tool_client_dispatches_odds_and_fixtures PASSED [41%]

tests/test_news_context.py::test_tool_dispatcher_routes_calls_and_serialises_results PASSED [44%]

tests/test_news_context.py::test_seed_context_queries_vector_repo_for_both_teams PASSED [46%]

tests/test_news_context.py::test_should_stop_requires_enough_high_reliability_evidence PASSED [48%]

tests/test_news_context.py::test_news_context_agent_run_collects_and_saves_evidence PASSED [51%]

tests/test_news_context.py::test_news_context_agent_ignores_malformed_evidence_json PASSED [53%]

tests/test_news_context.py::test_news_context_agent_records_unknown_tool_error PASSED [55%]

tests/test_news_context.py::test_news_context_agent_requires_live_search_even_with_seeded_evidence PASSED [58%]

tests/test_news_context.py::test_news_context_agent_no_longer_persists_evidence_during_research PASSED [60%]

tests/test_news_context.py::test_extract_evidence_suppresses_conflicting_market_direction

```
s PASSED [ 62%]
tests/test_prompt_loader.py::test_render_news_context_research_messages_includes_dynamic_fields PASSED [ 65%]
tests/test_prompt_loader.py::test_render_news_context_evidence_messages_includes_constraints PASSED [ 67%]
tests/test_prompt_loader.py::test_render_synthesis_decision_messages_uses_yaml_prompt PASSED [ 69%]
tests/test_sqlite_repository.py::test_sqlite_repository_round_trips_match_forecast_and_evidence PASSED [ 72%]
tests/test_sqlite_repository.py::test_sqlite_repository_updates_match_status_and_uses_seeded_source_scores PASSED [ 74%]
tests/test_sqlite_repository.py::test_sqlite_repository_returns_recent_finished_matches_for_team PASSED [ 76%]
tests/test_stats_market_agent.py::test_stats_market_agent_populates_matches_odds_and_baselines PASSED [ 79%]
tests/test_stats_market_agent.py::test_stats_market_agent_records_error_when_odds_missing PASSED [ 81%]
tests/test_stats_market_agent.py::test_stats_market_agent_uses_recent_finished_history_for_baseline PASSED [ 83%]
tests/test_synthesis_alert.py::test_synthesis_alert_agent_sends_alert_when_guard_passes PASSED [ 86%]
tests/test_synthesis_alert.py::test_synthesis_alert_agent_withholds_alert_and_records_reasons_when_guard_fails PASSED [ 88%]
tests/test_synthesis_alert.py::test_draw_positive_evidence_increases_draw_share PASSED [ 90%]
tests/test_synthesis_alert.py::test_single_market_evidence_is_discounted_vs_both_market_evidence PASSED [ 93%]
tests/test_synthesis_alert.py::test_llm_conviction_influences_confidence_score PASSED [ 95%]
tests/test_team_name_normalizer.py::test_team_name_normalizer_maps_known_aliases_to_canonical_names PASSED [ 97%]
tests/test_team_name_normalizer.py::test_team_name_normalizer_returns_trimmed_input_for_unknown_name PASSED [100%]

===== 43 passed in 4.52s =====
```

11 · Conclusion

The Soccer Forecast Agent is a fully operational multi-agent AI system built to SOLID principles:

Capability	Implementation
Statistical forecasting	Dixon-Coles MLE model, beats random on winner Brier (0.220 vs 0.222)
Qualitative evidence	ReAct loop: ChromaDB seed → Tavily web search → structured EvidenceItem extraction
LLM adjudication	Bounded log-odds adjustment via synthesis decision JSON
Alert quality bar	AlertGuard: 6 hard checks (evidence count, reliability, diversity, edge, confidence, spam)
Styled alerts	HTML + plain-text dual-part email via SMTP / ConsoleAlertChannel fallback

Capability	Implementation
Persistence	SQLite (fixtures, forecasts, evidence) + ChromaDB (514 article chunks, 20 teams)
Extensibility	LLM provider, alert channel, and baseline strategy are all swappable via env var
Observability	Structured logging, Brier-score backtest, weekly scheduled refresh script

Production roadmap (Phase 9): xG-powered Dixon-Coles, Pinnacle line-movement signal, lineup-aware parameter adjustment, Telegram bot alerts, rolling auto-calibration, and cloud deployment on a \$15/month VPS.

This project is an academic decision-support assistant. All alerts carry a conservative framing disclaimer and do not constitute financial advice.